ASSIGNMENT 2 – CO2

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

1. Challenge: Hostel Fee Concession Updater (Three Implementations)

To demonstrate why only reference and pointer parameters mutate the caller's stored fee, I implemented three versions for Merit (10%), Sports (15%), and Single Parent (25%) concessions on an initial fee of Rs. 50,000.

```
void concessionByValue(long long fee, int percent) {
    fee -= fee * percent / 100; // modifies local stack copy only
}
void concessionByRef(long long& fee, int percent) {
    fee -= fee * percent / 100; // modifies caller's variable via alias
}
void concessionByPtr(long long* fee, int percent) {
    if (fee != nullptr) *fee -= (*fee) * percent / 100; // dereferences caller's address
}
```

Method	Predicted Caller Fee	Observed Caller Fee	Parameter Storage
Call by Value (10%)	50000	50000	Separate copy in callee stack frame
Call by Reference (15%)	42500	42500	Alias bound directly to caller's variable
Call by Pointer (25%)	37500	37500	Local pointer holding caller's address

2. Inquiry & Design Decision

- **Why only two change the caller's fee:** Call by value creates an independent copy on the function's stack frame, so changes vanish on return. Call by reference binds an alias to the caller's memory, and call by pointer stores the caller's address and mutates it via *fee.
- **When to use const reference:** Use const T& when reading large objects (such as a Student record) without copying them and while guaranteeing the function cannot modify the caller's data.
- **Design Decision (In-place vs Return New Fee):** When several concessions may apply to one student, **returning the new fee** is safer and clearer than mutating in place. Returning a value makes each concession explicit (fee = applyConcession(fee, 10);), prevents accidental double-application, and allows computing quotes without overwriting the stored base fee.

3. Predict the Output (Written Before Running)

> **Pre-Execution Prediction vs Observed Output**
>
> **Predicted Output:** 1 12 13 (line 1), 1 100 (line 2), 16 13 6 (line 3).
> **Observed Output:** 1 12 13, 1 100, 16 13 6 (Zero differences — exact match).

- change(a, b, &c): a stays 1 (by value), b becomes 12 (by reference), and c becomes 13 (by pointer).
- bigger(a, c) = 100: Since c (13) > a (1), bigger returns a reference to c, so c is assigned 100 while a stays 1.
- Default arguments in show(x, y=5, z=10): show(1) uses y=5, z=10 (16); show(1, 2) uses z=10 (13); show(1, 2, 3) uses no defaults (6).

4. Complete the Code (TODO 1)

```
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
```

Observed Output: after call by value : 45000 and after call by reference: 40500. Call by value leaves hostelFee at 45000 because only the local copy of fee is reduced.

5. Alternative Concept & Complexity Analysis

Returning the new fee uses long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; }. All versions run in **O(1)** time (single arithmetic update). Space copied per call differs sharply when scaling from a primitive to a large Student object (e.g., 256 bytes):

Passing Mechanism	Time Complexity	Space Copied (long long)	Space Copied (256 B Student)
By Value	O(1)	8 bytes	256 bytes (full copy)
By Reference (&)	O(1)	8 bytes (address)	8 bytes (address)
By Const Reference (const &)	O(1)	8 bytes (address)	8 bytes (read-only)

Q2 – Return by Reference and Its Dangers (4 Marks)

1. Challenge: Examination Marks Register

```
int& highestMark(int marks[], int n) {
    int maxIdx = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[maxIdx]) maxIdx = i;
    return marks[maxIdx];
}
int& dangerousLocalRef() {
    int localMark = 95;
    return localMark; // Dangling reference!
}
```

- **L-value modification:** Because highestMark(marks, 5) returns an int& bound to marks[2], it can appear on the left of += or =: highestMark(marks, 5) += 2; updates 91 to 93, and highestMark(marks, 5) = 100; sets it to 100.

- **Compiler Warning & Undefined Behaviour:** Compiling with g++ -std=c++17 -Wall -Wextra emits: warning: reference to local variable 'localMark' returned [-Wreturn-local-addr]. When dangerousLocalRef() returns, its stack frame is destroyed; reading or writing through the returned reference accesses invalid stack memory (Undefined Behaviour).

Return Type	Can Modify Caller Element?	Copies Value?	Typical Use Case
int& (Reference)	Yes (valid l-value)	No	In-place updates like += 2
const int& (Const Ref)	No (read-only)	No	Zero-copy read of large objects
int (By Value)	No (r-value copy)	Yes (4 bytes)	Safe return of local results

2. Changed Condition & Alternative Concept Analysis

Vector Growth Dangling Reference: If marks are stored in a std::vector<int> and push_back() exceeds capacity, the vector allocates a new buffer, moves elements, and frees the old buffer. In my test run, vec[1] moved from 0x56b6b72e7ec4 to 0x56b6b72e7ee4, making any reference obtained before growth dangle.

Return Style	Calling Code for +2	Time / Space	Safety on Vector Growth
Reference (int&)	highestMark(v) += 2;	O(n) / O(1)	Unsafe: invalidates on reallocation
Pointer (int*)	*highestMarkPtr(v) += 2;	O(n) / O(1)	Unsafe: address invalidates on growth
Index (int)	v[highestMarkIdx(v)] += 2;	O(n) / O(1)	**Safe:** index stays valid after reallocation

Q3 – Default Arguments and Function Overloading (6 Marks)

1. Ambiguity Error Diagnosis

Declaring charge(double), charge(double, bool), and charge(double, bool = false, bool = false) causes a compile error: error: call of overloaded 'charge(double)' is ambiguous. Calling charge(2.5) matches both the 1-parameter overload and the 3-parameter overload (using two defaults), and charge(2.5, true) matches both the 2-parameter and 3-parameter overloads.

2. Redesigned Unambiguous Functions & 6 Calls

```
double charge(int grams, bool express = false, bool insured = false);
double charge(double kg, bool express = false, bool insured = false);
double charge(const Parcel& p);
```

Function Call	Predicted Overload	Observed Selection	Resolution Rule
charge(2)	charge(int, bool, bool)	charge(int grams...)	Exact match (int)

Function Call	Predicted Overload	Observed Selection	Resolution Rule
charge(2.0f)	charge(double, bool, bool)	charge(double kg...)	Promotion (float → double)
charge(2.5)	charge(double, bool, bool)	charge(double kg...)	Exact match (double)
charge(2.5, true)	charge(double, bool, bool)	charge(double kg...)	Exact match (2 args)
charge(1500, true, true)	charge(int, bool, bool)	charge(int grams...)	Exact match (3 args)
charge(p)	charge(const Parcel&)	charge(const Parcel&)	Exact match (Parcel)

3. Inquiry & Alternative Concept Analysis

- **Overload Resolution Order:** The compiler ranks candidates by: (1) Exact match, (2) Integral/floating promotion (e.g., float to double), (3) Standard conversion (e.g., int to double), and (4) User-defined conversion.
- **Right-to-left Default Arguments:** Positional arguments bind left-to-right; if a middle parameter had a default without trailing ones, the compiler could not tell which parameter an omitted argument refers to.
- **Alternative (Single Parcel Struct):** Using one charge(const Parcel&) with default member initializers eliminates multiple overloads. Overload resolution occurs purely at **compile time** (O(1) / zero runtime cost), copies only an 8-byte reference per call, and scales cleanly when adding new options (like fragile handling) without changing function signatures.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

1. Class Specification (Inside & Outside Member Definitions)

```
class SavingsAccount {
private:
    std::string accountNumber;
    double balance;
    double transactions[10];
    int count, startIdx;
    void recordCircular(double amount);
public:
    SavingsAccount(const std::string& accNo, double initialBalance)
        : accountNumber(accNo), balance(initialBalance), count(0), startIdx(0) {}
    double getBalance() const { return balance; } // defined inside class
    void deposit(double amount);                  // defined outside via ::
    bool withdraw(double amount);                 // defined outside via ::
    void miniStatement() const;                   // defined outside via ::
};
```

2. Inquiry, Design Decision & Alternative Analysis

- **Why the array must be private:** If transactions or count were public, external code could corrupt the transaction history, desynchronize balance from recorded amounts, or write out-of-bounds indices.
- **11th Transaction (Shifting vs Circular Index):** Shifting moves elements 1..9 left into 0..8 and writes the new entry at index 9, costing **O(k)** (9 moves). A circular index overwrites transactions[startIdx] and advances startIdx = (startIdx + 1) % 10 in **O(1)** time.
- **Scaling to 50 Transactions:** The circular-index design changes least: updating a single constant MAX_TX = 50 scales capacity immediately while keeping insertion at O(1).

Implementation	Insert 11th+ Transaction	Space Used	Trade-off
Shifting Fixed Array	O(k) element shifts	O(k) stack/object	Slow as k grows; contiguous order
Circular Buffer (Start Index)	**O(1)** modulo write	O(k) stack/object	Fastest; zero heap allocation
std::deque (push_back/pop_front)	O(1) amortized	O(k) + heap blocks	Flexible size; slight heap overhead

Q5 – Static Members and Arrays of Objects (7 Marks)

1. Class Design & Inquiry

- **Static Members & Definition:** issued (shared seat counter) and CAPACITY (shared bus limit) must be static, along with static int count(). Non-const static data members must be defined once outside the class: int BusTicket::issued = 0;.

- **Why static functions lack this:** A static member function belongs to the class as a whole and can be called via BusTicket::count() without any object instance, so no this pointer exists and non-static members cannot be accessed.
- **Arrays of Objects & Loop/Function Persistence:** BusTicket trip[4] = {BusTicket("Asha"), ...}; constructs 4 elements sequentially. Creating tickets inside a loop or calling a booking function twice continues incrementing the same BusTicket::issued variable across calls.

2. Predict the Output (Token Class)

> **Pre-Execution Prediction vs Observed Output**
> **Predicted:** 101 103 104 4 | **Observed:** 101 103 104 4 (Exact match).

Token::next is initialized to 100 so IDs start at 101 while next - 100 yields the exact count of issued tokens. The constructor ran **4 times** (3 for counter[3], 1 for extra).

3. Complete the Code (BusTicket TODO 1 & TODO 2)

```
explicit BusTicket(const std::string& n) : seat(0), name(n) {
   if (issued < CAPACITY) seat = ++issued; // TODO 1
}
static int count() { return issued; }       // TODO 2
```

Observed Output: Asha: Seat 1, Bala: Seat 2, Chitra: Seat 3, Dev: Waiting, Issued: 3. Calling a 2-ticket booking function twice produced Issued: 2 after Call 1 and Issued: 3 (with the 4th passenger on Waiting) after Call 2.

4. Alternative Concept & Complexity Analysis

Counter Design	Time / Space	Encapsulation	Two Buses at Once?
static int issued	O(1) / 1 int	Strong (private to class)	No (shared across all buses)
Global Variable	O(1) / 1 int	None (modifiable anywhere)	No (single global state)
TicketOffice Object	O(1) / 1 int per bus	Strong (member of office)	**Yes** (one office per bus)

Q6 – Friend Functions and Inline Functions (8 Marks)

1. Challenge, Inquiry & Complete the Code

- **Why friend is needed:** Operations like minutesBetween(a, b) and earlier(a, b) treat both Time objects symmetrically. A non-member friend accesses private h and m directly while keeping both arguments symmetric.
- **Friend vs Member:** A member function is called as a.isEarlier(b) with an implicit this; a friend is called as earlier(a, b) with no this pointer. Making h, m public would destroy encapsulation and allow invalid states like h = 99.

- **inline Keyword:** Requests the compiler to substitute the function body at the call site to avoid call/return stack frame overhead when called millions of times.

```
int minutesBetween(const Time& a, const Time& b) {
    int t1 = a.h * 60 + a.m, t2 = b.h * 60 + b.m;
    return (t1 > t2) ? (t1 - t2) : (t2 - t1);
}
bool earlier(const Time& a, const Time& b) {
    return (a.h < b.h) || (a.h == b.h && a.m < b.m);
}
```

Correctness Checks: 9:45 vs 13:10 → 205 false; across noon (11:50 vs 12:15) → 25 mins, earlier=true; same time twice (12:00 vs 12:00) → 0 mins, earlier=false.

2. Growth Investigation (10,000,000 Calls at -O0 and -O2)

Build Configuration	Optimization	Measured Time	Explanation
Without inline	-O0	68.85 ms	Full call/return stack frame per call
With inline	-O0	69.30 ms	-O0 ignores inlining hints
Without inline (noinline)	-O2	28.56 ms	Optimized body, still pays call overhead
With inline	-O2	**20.51 ms**	**~28% faster:** body inlined into loop

Deployment Decision: Keep the small inline friend functions. They run in O(1) at the same speed as inline member functions after optimization, preserve strict encapsulation of h and m (unlike public getters), and keep symmetric syntax.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks)

A. Diagnose the Limitation

In **Q4 (SavingsAccount Mini-Statement)**, the shifting-array design stores the last K transactions in a fixed array and shifts K - 1 elements left on every new transaction once full. That makes each insertion **O(K)** (and **O(N · K)** over N transactions) and hardcodes K = 10 at compile time.

B. Define the Design Objective

Achieve strictly **O(1)** worst-case time per deposit()/withdraw() insertion regardless of history size K, and support configurable capacity K per account without using any standard library container (std::deque/std::vector).

C. Design the Revised Solution

I redesigned SavingsAccount as SavingsCircular using a custom dynamically allocated **circular ring buffer** (double* tx) with cap, head (oldest index), and count, plus full Rule-of-Three memory management (destructor, copy constructor, copy assignment):

```
Class SavingsCircular:
  Private:
    double* tx       // heap-allocated ring buffer of size cap
    int cap, head, count
    double balance
  Public:
    SavingsCircular(capacity, initBal)
    ~SavingsCircular(), copy constructor, copy assignment operator=
    void deposit(amt)     // O(1): tx[head] = amt; head = (head + 1) % cap
    double getNth(i)      // O(1): return tx[(head + i) % cap]
```

D. Predict Before Implementing

Because the circular buffer eliminates the O(K) shift loop, I predict an **O(1)** insertion time that stays nearly constant as K grows, yielding at least a **10x speedup** over the shifting array for K = 1,000 and N = 100,000 insertions under -O2.

E. Implement and Validate the Redesign

Test / Metric	Original (Shifting Array)	Redesigned (Ring Buffer)	Comparison
Correctness (K=10, 15 tx)	600 .. 1500	600 .. 1500	EXACT MATCH
Stress Benchmark (K=1000, N=100,000, -O2)	5.573 ms	**0.378 ms**	**14.73x faster**

F. Defend the Final Design

The objective was achieved: insertion time dropped from O(K) to O(1) with a measured **14.73x speedup** at K = 1,000. The new costs are manual heap management (requiring the Rule of Three to avoid leaks/double-frees), modulo index arithmetic when printing, and one extra heap allocation at construction. The original fixed shifting array remains preferable only for tiny fixed buffers ($K \leq 10$) where stack allocation and contiguous chronological layout outweigh asymptotic speed.